

STELA: Spatiotemporal Forecasting via Graph Learning and Entropy-Guided LLM Adaptation

Tiantian Huang
School of Cyber Engineering
Xidian University
Xi'an, China
24151213672@stu.xidian.edu.cn

Yue Li*
School of Cyber Engineering
Xidian University
Xi'an, China
liyue@xidian.edu.cn

Wei Shao
Data61, CSIRO
Clayton, VIC, Australia
UNSW
Sydney, Australia
wei.shao@data61.csiro.au

Ziqi Xu
RMIT University
Melbourne, Australia
ziqi.xu@rmit.edu.au

Qipeng Song*
School of Cyber Engineering
Xidian University
Xi'an, China
qpqsong@xidian.edu.cn

Hui Li
School of Cyber Engineering
Xidian University
Xi'an, China
lihui@mail.xidian.edu.cn

Abstract

Spatiotemporal forecasting underpins many web-based services, from city-wide traffic navigation to large-scale mobility platforms and infrastructure management. Recently, large language models (LLMs) have shown great potential for enhancing such tasks. However, enabling LLMs to understand structured spatiotemporal data remains challenging due to three persistent gaps: (i) a representation gap, arising from the mismatch between continuous graph signals and discrete token-based embeddings; (ii) a fusion gap, reflecting the difficulty of harmonizing heterogeneous temporal, spatial, and local cues; and (iii) an adaptation gap, concerning the efficient alignment between learned embeddings and the LLM's semantic space. To bridge these gaps, we propose STELA, a unified framework that integrates structured spatiotemporal modeling with LLM-based reasoning through a progressive alignment pipeline. STELA employs a spatial-aware transformer for cross-node temporal encoding, an adaptive spatiotemporal graph encoder for dynamic relations, a gated multi-path fusion module for feature integration, and an entropy-guided adapter that selectively fine-tunes low-entropy layers to align with the LLM's semantic space without full-model retraining. Experiments on multiple real-world benchmarks show that STELA consistently outperforms recent LLM-based forecasting methods and strong non-LLM baselines, demonstrating its effectiveness for spatiotemporal reasoning in urban environments. Our code is available at <https://github.com/KarryHuang12138/STELA-code>.

CCS Concepts

• Information systems → Spatial-temporal systems; World Wide Web; • Computing methodologies → Knowledge representation and reasoning.

*Corresponding author



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792634>

Keywords

Spatiotemporal forecasting, Representation learning, Large language models, Web-based services

ACM Reference Format:

Tiantian Huang, Yue Li, Wei Shao, Ziqi Xu, Qipeng Song, and Hui Li. 2026. STELA: Spatiotemporal Forecasting via Graph Learning and Entropy-Guided LLM Adaptation. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792634>

1 Introduction

Background. The rapid proliferation of web-connected sensors, mobile devices, and online services has led to massive growth in spatiotemporal data across domains such as transportation, mobility, and urban management. These data streams power emerging Web-of-Things (WoT) applications, including ride-hailing, bike-sharing, and intelligent traffic systems, that are reshaping modern smart cities [3, 34, 36, 38]. Accurate spatiotemporal prediction is thus vital for reliable and efficient web-based services [18, 28], supporting proactive traffic control, timely emergency response, and informed urban planning [11, 29].

Challenges. Early spatiotemporal forecasting models [12, 31] jointly captured spatial and temporal dependencies under fixed topologies. Later works introduced attention mechanisms [6, 43] and adaptive graph learning [1, 9, 33], improving temporal expressiveness and spatial adaptability. Recently, large language models (LLMs) have shown strong potential in time-series forecasting [2, 8, 26, 39], leveraging pretrained knowledge to uncover complex temporal patterns and event-driven relationships beyond conventional models. However, directly integrating spatiotemporal data into LLMs remains difficult: graph-structured signals are continuous and numerical, while LLMs operate in discrete semantic spaces, making naive fusion prone to structural inconsistency and semantic misalignment [4, 13, 23]. Addressing these issues requires bridging gaps in representation, fusion, and semantic adaptation, leading to three key challenges:

C1: Representation gap, how to represent spatiotemporal data in an LLM-compatible form. Spatiotemporal data are continuous, high-dimensional, and structurally entangled across

both time and space, whereas LLMs operate in discrete token-based semantic spaces. This fundamental mismatch makes it non-trivial to transform structured signals, such as traffic flow series or sensor readings, into representations interpretable by LLMs. Directly injecting such embeddings disrupts the learned linguistic geometry of LLMs, leading to representation misalignment and degraded reasoning. The challenge, therefore, is to construct a transformation that preserves the temporal and spatial continuity of the original data while mapping it into a latent space that can be seamlessly integrated into the LLM’s processing pipeline.

C2: Fusion gap, how to unify temporal, spatial, and local signals before semantic reasoning. Even after being encoded into continuous embeddings, temporal and spatial signals remain heterogeneous in both granularity and semantics. Temporal dynamics describe node-wise evolution over time, while spatial relations encode inter-node dependencies, and local fluctuations capture short-term irregularities. Without proper alignment, these multi-source signals provide the LLM with inconsistent cues, preventing coherent reasoning over spatiotemporal contexts. The challenge is thus to design an effective fusion mechanism that harmonizes these complementary sources, enabling node-level adaptability and semantic consistency across multiple granularities of information.

C3: Adaptation gap, how to align fused spatiotemporal embeddings with the LLM’s semantic space. Although the fused representations capture spatiotemporal structure, they still reside in a continuous embedding space that remains misaligned with the LLM’s internal semantic space. Meanwhile, fully fine-tuning the entire LLM to adapt to these inputs is computationally prohibitive, while freezing it limits its ability to internalize new structural knowledge. The key challenge is to achieve an efficient and interpretable alignment strategy, enabling the LLM to semantically understand the spatiotemporal embeddings while preserving general reasoning ability and scalability.

Method. To address these challenges, we propose STELA, a unified framework that bridges structured spatiotemporal modeling with LLM-based semantic reasoning. The key novelty of STELA is the *progressive alignment* design that makes these components work coherently to bridge the representation, fusion, and adaptation gaps. STELA is designed around this progressive alignment pipeline: it first transforms raw temporal sequences and spatial structures into LLM-compatible latent embeddings via a *spatial-aware transformer encoder* and an *adaptive spatiotemporal graph encoder*, then harmonizes heterogeneous temporal, spatial and local signals through a *multi-path fusion mechanism*, and finally aligns the fused representations with the LLM’s semantic space using an *entropy-guided adapter*. This modular architecture enables STELA to preserve structured dependencies while injecting high-level semantic knowledge from LLMs, achieving efficient and interpretable reasoning across temporal, spatial and semantic aspects.

Contributions. Our main contributions are as follows:

- We propose STELA, a unified end-to-end framework that integrates structured spatiotemporal modeling with LLM-based semantic reasoning. STELA employs a progressive alignment pipeline that transforms, fuses, and aligns spatiotemporal representations, enabling LLMs to reason over structured signals without handcrafted textual supervision.
- We design a representation–fusion–adaptation architecture that addresses key gaps in integrating LLMs with spatiotemporal data. It includes a spatial-aware transformer and an adaptive graph encoder for LLM-compatible representation learning, a multi-path gating mechanism for spatiotemporal fusion, and a novel entropy-guided adapter for efficient alignment between continuous embeddings and the LLM’s semantic space.
- We conduct extensive experiments on diverse real-world benchmarks, showing that STELA consistently outperforms state-of-the-art spatiotemporal forecasting baselines and recent LLM-enhanced models in accuracy, generalization, and robustness. Ablation and visualization analyses further confirm the interpretability and scalability of our framework.

2 Related Work

2.1 Spatiotemporal Graph Forecasting

Spatiotemporal graph neural networks (STGNNs) jointly model spatial dependencies among nodes and temporal dynamics in sequential data. Representative models such as STGCN [35], DCRNN [12], and Graph WaveNet [31] typically rely on predefined adjacency matrices derived from physical or distance relationships, combining graph convolutions with recurrent or convolutional temporal encoders. While effective in capturing local spatial correlations, fixed connectivity and predominantly node-wise temporal encoding make it difficult to represent evolving historical interactions across nodes. Subsequent works improve temporal modeling with attention mechanisms [6, 43] or multi-scale convolution [32], extending temporal receptive fields [42]. However, many methods still encode each node’s temporal sequence largely independently, leading to limited cross-node temporal propagation and reduced ability to adapt to time-varying spatial dependencies. These limitations motivate models that can capture cross-node temporal dependencies under evolving spatial structures.

2.2 Adaptive and Dynamic Graph Learning

To alleviate the rigidity of static topologies, adaptive graph learning has become a key direction in spatiotemporal modeling. AGCRN [1] learns node embeddings end-to-end to generate adaptive adjacency matrices, removing the need for predefined graphs. Building on this idea, MTGNN [32] and DGCRN [10] introduce data-driven graph construction modules that infer spatial dependencies from evolving time-series patterns, while dynamic architectures such as DSTAGNN [9] and DGWNet [33] further model time-varying relations via temporal attention or multiple learned adjacencies. Despite these advances, many methods combine static priors and learned graphs mainly at global or layer granularity, effectively applying uniform balancing across nodes and time steps. Dual-adaptive and multi-graph approaches [22, 24] improve flexibility, yet still fall short of fine-grained, node-aware and temporally sensitive balancing, which can harm local stability under continuous structural evolution.

2.3 Transformer and LLM-Based Forecasting

Transformers model long sequences via self-attention [20, 30, 44], capturing global temporal dependencies with scalable parallelism. However, they are often spatially agnostic and typically rely on

external priors to preserve structural coherence. Recently, large language models (LLMs) have been adapted for time-series forecasting by tokenizing numerical data and processing them as sequential “language” [2, 8, 26, 39]. While LLMs provide strong semantic abstraction and cross-domain reasoning, they may neglect graph connectivity, yielding semantically coherent but spatially inconsistent forecasts. To mitigate this issue, hybrid approaches combine LLM reasoning with graph-based encoders or convolutional tokenizers [4, 13, 23, 37]. Nevertheless, many methods still fuse structural and semantic signals at a global level or with static weighting, lacking node-wise and temporally aware alignment, and the attention cost further challenges scalability in large, heterogeneous, and evolving transportation systems.

3 Preliminaries

In this section, we formalize the key concepts and notations used throughout the paper. We begin by defining the spatiotemporal graph structure and node-level observations, followed by the latent semantic context that influences system dynamics. Finally, we describe the forecasting objective that integrates these components into a unified formulation.

Definition 3.1 (Spatiotemporal Graph). A spatiotemporal system is represented as a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V} = \{v_1, v_2, \dots, v_N\}$ denotes the set of N nodes (e.g., sensors, stations, or regions), and $\mathcal{E}$ represents spatial or functional relationships among them. Each graph is associated with an adjacency matrix $\mathbf{A}_t \in \mathbb{R}^{N \times N}$, which captures two complementary types of dependencies: (1) static dependencies: relatively stable spatial or structural connections (e.g., road topology or physical proximity), represented by a base adjacency matrix $\mathbf{A}^{(s)}$; (2) dynamic dependencies: time-varying or data-driven relations that evolve with system states, represented by a time-dependent matrix $\mathbf{A}^{(d)}$.

Definition 3.2 (Node Features and Input Tensor). At each time step t , each node $v_i \in \mathcal{V}$ is associated with a feature vector $\mathbf{x}_t^{(i)} \in \mathbb{R}^F$, where F denotes the feature dimensionality (e.g., traffic speed, passenger demand, or power usage). The observations collected from all nodes over a historical window of length T form the input tensor

$$\mathbf{X}_{t-T+1:t} = [\mathbf{x}_{t-T+1}, \dots, \mathbf{x}_t] \in \mathbb{R}^{N \times T \times F}.$$

This tensor serves as the primary observable input for the forecasting model.

Definition 3.3 (Semantic Context). We denote by $\mathcal{S}$ a set of latent contextual factors that influence system dynamics, such as holidays, weather variations, and unexpected disruptions. Although these signals are not explicitly included in the input tensor $\mathbf{X}_{t-T+1:t}$, they implicitly affect mobility or demand patterns. In our framework, such contextual semantics are captured through external knowledge integration and large language model representations during training.

Definition 3.4 (Spatiotemporal Forecasting). Given the historical observations $\mathbf{X}_{t-T+1:t}$, the dynamic graph sequence $\{\mathcal{G}_\tau\}_{\tau=t-T+1}^t$, and the latent semantic context $\mathcal{S}$, the spatiotemporal forecasting task aims to predict the future node features over the next H steps:

$$\hat{\mathbf{X}}_{t+1:t+H} = f(\mathbf{X}_{t-T+1:t}, \{\mathcal{G}_\tau\}_{\tau=t-T+1}^t, \mathcal{S}),$$

where $f(\cdot)$ is a learnable forecasting function that jointly captures temporal evolution, spatial dependencies, and semantic influences in a unified manner.

4 Methodology

4.1 Overview of Framework

STELA is an end-to-end modular framework that bridges continuous spatiotemporal signals with the discrete semantic space of LLMs (Fig. 1). It progressively maps raw observations to LLM-compatible embeddings through four stages. **First**, the *Spatial-Aware Transformer Encoder* (Sec. 4.2) summarizes each node’s temporal history while injecting cross-node attention to capture correlated spatial dynamics. **Second**, the *Adaptive Spatiotemporal Graph Encoder* (Sec. 4.3) builds a hybrid graph by combining physical topology with data-driven interactions via node-wise gating. **Third**, the *Multi-Path Feature Fusion* module (Sec. 4.4) integrates temporal and structural embeddings, including a local convolutional branch for fine-grained continuity, yielding a unified representation suitable for LLM reasoning. **Finally**, the *Entropy-Guided LLM Adapter* (Sec. 4.5) aligns the fused embeddings to the LLM semantic space by selectively fine-tuning low-entropy layers, followed by a lightweight prediction head for forecasting. Together, these components implement a progressive spatiotemporal–semantic bridging pipeline; algorithmic details are provided in Appendix A.

4.2 Spatial-Aware Transformer Encoder

The first component of our framework is a Transformer-based encoder designed to convert raw temporal signals at each node into compact, spatially contextualized embeddings that are compatible with large language models. Its goal is to summarize temporal variations while maintaining awareness of how different nodes influence each other within the observation window. This module consists of two core parts: a learnable temporal projection for sequence abstraction, and a cross-node self-attention mechanism enhanced with spatial embeddings.

(1) *Temporal projection.* For a given node v_n , we denote its historical observation window as $\mathbf{X}_n \in \mathbb{R}^{T \times F}$, where T is the number of past time steps and F the feature dimension. In traffic forecasting, F is usually small (often $F = 1$), corresponding to flow, speed, or occupancy. Even when $F > 1$, features can be treated as a unified vector per time step or aggregated through light-weight pooling, so we regard $\mathbf{X}_n$ as a sequence of T scalar-like observations for clarity.

We apply a learnable linear projection along the temporal axis:

$$\tilde{\mathbf{x}}_n = \mathbf{W}_t \mathbf{X}_n, \quad \mathbf{W}_t \in \mathbb{R}^{d \times T}, \quad \tilde{\mathbf{x}}_n \in \mathbb{R}^d. \quad (1)$$

This projection performs data-driven temporal compression, mapping each node’s T -step trajectory into a d -dimensional latent vector. Unlike handcrafted pooling (e.g., average or max pooling), which enforces fixed aggregation rules, the learnable projection $\mathbf{W}_t$ adapts to data statistics and learns an optimal weighting over temporal steps. Intuitively, it functions as a trainable temporal filter that extracts the most informative patterns from each node’s history. This step transforms the original time series into a compact latent representation, an essential abstraction before the model can inject semantic reasoning via the LLM.

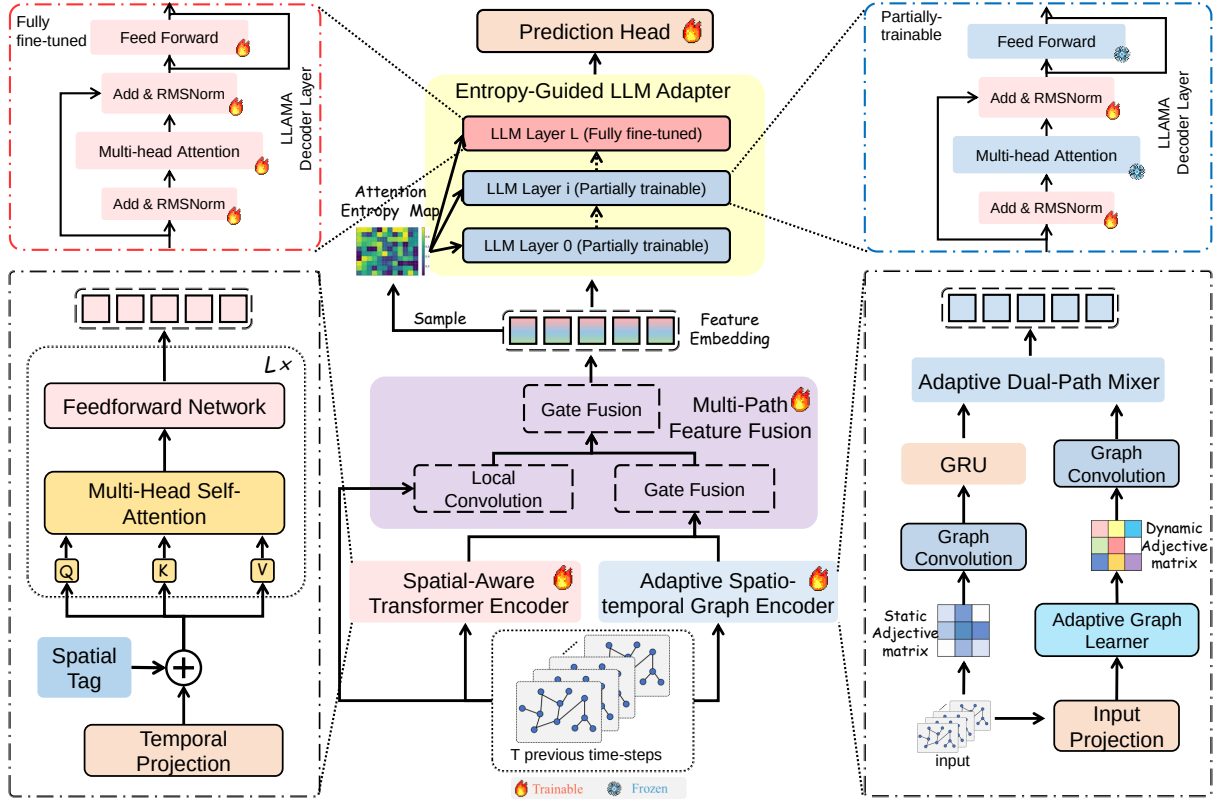


Figure 1: Overview of the STELA framework.

(2) *Cross-node self-attention with spatial embeddings.* After temporal projection, each node's latent representation $\tilde{x}_n$ captures only its own temporal behavior. To enable relational reasoning across nodes, we introduce a learnable spatial embedding $e_n \in \mathbb{R}^d$ for each node v_n , which encodes its structural identity (e.g., geographical location or sensor role). By adding these embeddings, the model can distinguish node-specific semantics and establish spatial correspondence: $H^{(0)} = [\tilde{x}_1 + e_1, \tilde{x}_2 + e_2, \dots, \tilde{x}_N + e_N]^T \in \mathbb{R}^{N \times d}$.

We then apply L_s layers of Pre-LN Transformer blocks along the node dimension to model cross-node dependencies:

$$\tilde{H}^{(l)} = \text{LN}(\text{MSA}(H^{(l-1)})) + H^{(l-1)}, \quad (2)$$

$$H^{(l)} = \text{LN}(\text{MLP}(\tilde{H}^{(l)})) + \tilde{H}^{(l)}, \quad l = 1, \dots, L_s, \quad (3)$$

where LN, MSA, and MLP denote layer normalization, multi-head self-attention, and a feed-forward network, respectively. The multi-head attention is computed as:

$$\text{MSA}(H) = \text{softmax}\left(\frac{(HW_Q)(HW_K)^T}{\sqrt{d_h}}\right)(HW_V), \quad (4)$$

with $W_Q, W_K, W_V \in \mathbb{R}^{d \times d}$ learnable, and $d_h = d/h$ the per-head dimension for h heads. After stacking L_s such layers, we obtain the spatial-aware representation $H^{(t)} \in \mathbb{R}^{N \times d}$.

While this module structurally resembles the temporal encoder core in prior spatiotemporal networks, its purpose here is conceptually distinct. Instead of directly outputting forecasts, the spatial-aware Transformer acts as a “semantic abstraction layer”, transforming raw, continuous signals into structured latent tokens that encode both temporal continuity and spatial semantics. These embeddings provide the LLM with interpretable, structured inputs, forming the first step in bridging the gap between numerical spatiotemporal data and the discrete semantic space of large language models.

4.3 Adaptive Spatiotemporal Graph Encoder

While the temporal encoder in Section 4.2 focuses on summarizing each node's local evolution, the graph encoder complements it by modeling the structural dependencies among nodes. These two modules operate in parallel, respectively forming temporal and spatial representation, whose outputs are later fused for joint reasoning. The role of this module is therefore to construct a spatial embedding space that reflects both stable topological priors and dynamic relational variations, providing the LLM with interpretable structural information for subsequent semantic reasoning.

To achieve this, we design an *adaptive dual-path graph encoder* composed of two complementary pathways: a static structural path that captures stable physical dependencies, and a dynamic adaptive

path that learns transient interaction patterns directly from data. A lightweight gating mechanism then balances these two signals for each node, ensuring that the spatial representation is both stable and adaptive under varying conditions.

(1) *Static structural pathway.* For stable spatial dependencies, we follow the temporal graph convolutional formulation in [41], which integrates graph convolution with a gated recurrent unit to jointly capture spatial diffusion and temporal continuity. This process encodes physically constrained connectivity (e.g., road links or station layouts) into evolving node states. The resulting output is denoted as $H^{(s)} \in \mathbb{R}^{N \times d}$, representing the static-path embedding at time step t .

(2) *Dynamic adaptive pathway.* While static graphs offer interpretability, they cannot capture transient correlations emerging from evolving traffic conditions, events, or disruptions. To model these short-term and heterogeneous relations, we introduce a learnable graph that adaptively adjusts connectivity according to observed data.

Two trainable node embedding matrices, $E_1 \in \mathbb{R}^{N \times r}$ and $E_2 \in \mathbb{R}^{r \times N}$, are used to construct a data-driven adjacency:

$$A^{(d)} = \text{softmax}(\text{ReLU}(E_1 E_2)) \in \mathbb{R}^{N \times N}. \quad (5)$$

This operation implicitly encodes the interaction strength between nodes based on learned embeddings rather than fixed spatial distance. We first project each node's temporal history into a d -dimensional feature:

$$\tilde{X} = X W_p, \quad W_p \in \mathbb{R}^{T \times d}, \quad (6)$$

and then diffuse information through the adaptive graph:

$$H^{(d)} = (A^{(d)} \tilde{X}) W_g, \quad W_g \in \mathbb{R}^{d \times d}. \quad (7)$$

Unlike the static path, this adaptive path enables context-dependent message propagation that reflects real-time correlations between nodes.

(3) *Adaptive Dual-Path Mixer.* To integrate the complementary information from static and adaptive pathways, we design a node-wise gating mechanism that dynamically balances their contributions:

$$Z = [H^{(s)} \| H^{(d)}] \in \mathbb{R}^{N \times 2d}, \quad g = \sigma(Z W_m), \quad W_m \in \mathbb{R}^{2d \times 1}. \quad (8)$$

The final fused representation is computed as:

$$H^{(g)} = g \odot H^{(s)} + (1 - g) \odot H^{(d)}. \quad (9)$$

Here, $\sigma(\cdot)$ denotes the sigmoid function and $\odot$ is element-wise multiplication. This gating mechanism enables each node to autonomously determine whether to prioritize its static structural prior or its dynamically learned context, forming a balanced representation that is both interpretable and adaptive.

4.4 Multi-Path Feature Fusion

The temporal encoder focuses on node-specific temporal evolution, while the adaptive graph encoder emphasizes inter-node dependencies. Because different nodes may rely more heavily on one source of information than the other, a node-wise gating mechanism is introduced to adaptively balance their contributions.

(1) *Balancing temporal and spatial perspectives.* Let $H^{(g)} \in \mathbb{R}^{N \times d}$ be the output of the adaptive spatiotemporal encoder and $H^{(t)} \in \mathbb{R}^{N \times d}$ be the output of the spatial-aware transformer. We compute a gate:

$$G_1 = \sigma(H^{(t)} W_t + H^{(g)} W_g), \quad G_1 \in (0, 1)^{N \times d}, \quad (10)$$

and fuse the two representations as

$$H^{(st)} = G_1 \odot H^{(t)} + (1 - G_1) \odot H^{(g)}, \quad (11)$$

where $W_t, W_g \in \mathbb{R}^{d \times d}$ are learnable weights.

This operation performs *semantic interpolation* between temporal and spatial embeddings, ensuring that the fused representation remains within a unified latent space interpretable by the LLM. Unlike static concatenation or fixed averaging, the gate G_1 is computed independently for each node and feature dimension, dynamically assigning relevance to temporal or spatial cues based on current conditions. As a result, nodes dominated by stable spatial dependencies (e.g., intersections) and those influenced by strong temporal trends (e.g., highways) are adaptively weighted, allowing consistent representation learning across diverse contexts.

(2) *Local feature refinement.* To retain fine-grained fluctuations that may be smoothed out by high-level encoders, we extract a local feature $H^{(c)} \in \mathbb{R}^{N \times d}$ using a shallow convolutional operation. We then perform another gating step to merge this local representation with the global spatiotemporal context:

$$G_2 = \sigma(H^{(st)} W_s + H^{(c)} W_c), \quad G_2 \in (0, 1)^{N \times d}, \quad (12)$$

$$H^{(f)} = G_2 \odot H^{(st)} + (1 - G_2) \odot H^{(c)}. \quad (13)$$

This second gate acts as a learnable information bottleneck, selectively integrating global context and local variations to enhance representational sharpness without introducing redundant noise.

4.5 Entropy-Guided LLM Adapter

Integrating LLMs into spatiotemporal forecasting requires bridging a fundamental representational gap between numerical graph features and the semantic embedding space of the LLM. The fused representation $H^{(f)} \in \mathbb{R}^{N \times d}$ obtained from the previous modules encodes continuous structural and temporal dependencies, whereas LLMs operate in a discrete token-based semantic space that follows a distribution learned from large-scale textual dataset. Directly feeding $H^{(f)}$ into the LLM would lead to distributional mismatch and unstable attention behavior, preventing the model from interpreting structural correlations in a linguistically meaningful way.

To align these heterogeneous spaces, STELA introduces a lightweight semantic projection layer:

$$H^{(\text{align})} = H^{(f)} W_p + b_p, \quad W_p \in \mathbb{R}^{d \times d_{\text{LLM}}}. \quad (14)$$

This transformation performs (i) *dimensional alignment* by matching the latent dimension of $H^{(f)}$ to that of the LLM, and (ii) *semantic calibration* by reshaping the statistical distribution of spatiotemporal embeddings to fit the LLM's learned token manifold. In effect, the projection layer serves as a modality bridge, converting structured spatiotemporal representations into semantically interpretable vectors that can be processed through the LLM's attention pipeline. The aligned features $H^{(\text{align})}$ are then injected into the

decoder of a pretrained LLM to enable semantic reasoning over dynamic systems, such as identifying event-driven traffic anomalies or cross-region correlations.

However, directly fine-tuning the entire LLM remains computationally expensive, while freezing it discards adaptability [7]. Meanwhile, transformer layers exhibit unequal task sensitivity, and recent evidence suggests that *attention entropy* is a meaningful signal of task behavior: lower-entropy attention is more selective and often correlates with better context utilization and downstream performance [5, 14, 19, 21, 40]. Motivated by this, we introduce an entropy-guided selective adaptation strategy to decide *where* to adapt within a largely frozen LLM when aligning spatiotemporal graph embeddings to its semantic space. Concretely, during a warm-up stage on the downstream data, we compute the layer-wise average attention entropy under STELA-transformed inputs; layers with lower entropy exhibit more concentrated, task-relevant routing over the aligned tokens. We then unfreeze only the U lowest-entropy layers for fine-tuning, while keeping the remaining layers frozen except for lightweight modules such as normalization and embeddings [16, 17]. This yields higher utility per parameter under a fixed adaptation budget and preserves general semantic priors in the frozen majority of layers.

Formally, for layer l with attention matrix $\mathcal{A}^{(l)}$, the entropy is measured as

$$H_l = -\frac{1}{|\mathcal{A}^{(l)}|} \sum_{i,j} \mathcal{A}_{i,j}^{(l)} \log(\mathcal{A}_{i,j}^{(l)} + \epsilon), \quad (15)$$

where ϵ avoids numerical instability.

Given a total of L layers, the trainable subset is determined as

$$\mathcal{S} = \arg \min_{S \subset \{1, \dots, L\}, |S|=U} \{H_l \mid l \in S\}. \quad (16)$$

Only parameters in $\mathcal{S}$ are updated during training.

Let $x = H^{(\text{align})}$ denote the aligned embeddings. Passing x through the partially trainable LLM stack yields

$$\mathbf{H}^{(\text{LLM})} = \text{LLM}_{\text{partial}}(x) \in \mathbb{R}^{N \times d_{\text{LLM}}}, \quad (17)$$

producing task-aligned semantic embeddings at minimal overhead. This design allows STELA to unify numerical spatiotemporal reasoning with contextual semantic knowledge efficiently and adaptively.

Finally, the task-aligned semantic embeddings produced by the entropy-guided LLM adapter are fed into a lightweight node-wise linear head to generate multi-step forecasts. This design keeps the framework compact and stable while ensuring consistent predictions across all nodes. The entire STELA architecture is trained end-to-end with a mean absolute error (MAE) loss, offering robust and stable convergence under noisy spatiotemporal data.

5 Experiments

5.1 Experimental Setup

STELA uses LLaMA-7B [27] as its LLM backbone. All models are implemented in PyTorch and trained with Adam optimizer and early stopping. Evaluation follows common metrics: Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE). All experiments are conducted on NVIDIA A800 GPUs.

5.2 Datasets.

We use four real-world spatiotemporal datasets. **METR-LA** [12], **PEMS04**, and **PEMS08** [25] are benchmark traffic datasets collected at 5-minute intervals from loop detectors on California highways, covering different node counts (207, 307, and 170 respectively). **NYC-Subway** contains hourly turnstile entries at 167 stations across Queens and Brooklyn from Jul–Dec 2024, with graph edges defined by subway line topology. These datasets span diverse temporal resolutions and structural patterns, providing a broad testbed for spatiotemporal forecasting.

5.3 Baselines

We compare our method with a broad range of baselines, including statistical models (HA, ARIMA), spatiotemporal graph neural networks (TGCN [41], STGCN [35], DCRNN [12], Graph WaveNet [31], ASTGCN [6], AGCRN [1], MTGNN [32], DGCRN [10]), and recent LLM-based or hybrid methods (GATGPT [4], UrbanGPT [13], ST-LLM+ [15]). These baselines cover classical, neural, and generative paradigms, providing a comprehensive comparison.

Further implementation details, dataset statistics, and baseline descriptions are provided in Appendix B.

5.4 Overall Performance

As shown in Table 1, **STELA** consistently achieves the best performance across all datasets and horizons. Unlike recurrent or convolutional baselines that suffer from error accumulation and locality bias, STELA’s Transformer-based encoder builds global temporal correlations through cross-node attention, preserving long-range consistency in multi-step forecasting. Conventional adaptive GNNs (e.g., DGCRN, AGCRN) rely on fully learned graphs that easily fluctuate with noise, while our dual-path adaptive encoder anchors dynamic updates to a stable topology, yielding both robustness and flexibility. On *METR-LA*, STELA reduces RMSE by 8.1% compared with DGCRN, and the gain widens on long-horizon settings (e.g., 12h in *NYC_subway*), demonstrating stable temporal extrapolation. Moreover, simple feature concatenation in prior fusion designs often dilutes local variations, whereas STELA’s gated fusion adaptively balances temporal, structural, and short-term features for node-specific representation. Recent LLM-based models (e.g., UrbanGPT, ST-LLM+, GATGPT) integrate language reasoning with spatiotemporal data, but their structural modeling remains either static or loosely coupled with temporal dynamics, often resulting in spatially inconsistent forecasts. In contrast, the entropy-guided LLM adapter in STELA integrates high-level contextual semantics with dynamic spatiotemporal representations efficiently, enabling context-aware adaptation without overfitting. Together, these improvements explain why STELA maintains superior accuracy and generalization under diverse real-world conditions.

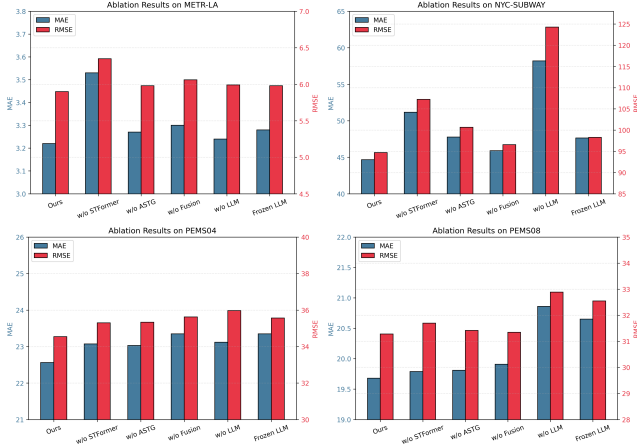
5.5 Ablation Study

To assess the contribution of each core component, we perform ablation experiments by selectively disabling key modules. The evaluated variants include: **(1) w/o STFormer**: removing the spatial-aware transformer encoder, where each node’s temporal dynamics are summarized by average pooling; **(2) w/o ASTG**: removing the adaptive spatiotemporal graph encoder, leaving the model without

Table 1: Forecasting performance on four datasets.

Method	METR-LA						NYC_subway					
	15min		30min		60min		3h		6h		12h	
	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE
HA	3.87	7.30	3.87	7.30	3.87	7.30	183.31	377.69	183.31	377.69	183.31	377.69
ARIMA	7.68	10.04	7.68	9.34	7.69	10.50	159.06	225.48	158.97	225.33	158.75	225.12
TGCN	3.55	5.59	4.07	6.50	4.84	7.84	75.02	124.92	105.65	162.26	124.00	214.71
STGCN	4.64	8.01	6.07	9.40	7.05	10.61	42.75	101.78	51.13	121.61	63.13	152.65
DCRNN	2.66	5.16	3.06	6.28	3.54	7.48	37.97	86.20	49.09	111.23	49.18	113.34
GWN	3.23	5.60	3.45	6.82	3.94	7.83	39.91	97.04	42.79	100.91	48.87	108.87
ASTGCN	3.23	6.37	3.70	7.41	4.34	8.65	58.08	139.22	65.71	150.99	61.73	149.83
AGCRN	3.05	5.52	3.58	6.75	4.25	8.03	46.12	102.87	50.64	110.79	56.72	135.33
MTGNN	2.96	5.41	3.42	6.61	4.08	7.89	43.88	99.24	48.36	107.52	55.47	133.12
DGCRN	2.74	5.10	3.21	6.25	3.83	7.68	39.81	90.74	46.03	102.01	51.22	121.85
GATGPT	2.98	5.56	3.47	6.72	4.15	8.05	42.68	100.83	47.89	109.65	56.92	136.70
UrbanGPT	2.38	4.12	2.73	4.89	3.35	6.12	33.45	67.10	44.60	89.32	46.10	98.75
ST-LLM+	2.90	5.53	3.39	6.69	4.05	8.02	41.86	102.61	46.94	111.66	56.64	137.56
STELA	2.23	4.01	2.60	4.78	3.22	5.90	31.25	64.69	42.07	86.24	44.67	94.49

Method	PEMS04						PEMS08					
	15min		30min		60min		15min		30min		60min	
	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE
HA	27.35	40.76	27.35	40.76	27.35	40.76	22.32	33.22	22.32	33.22	22.32	33.22
ARIMA	32.34	69.13	32.11	68.13	32.10	68.02	24.08	43.45	24.04	43.30	24.04	43.28
TGCN	23.28	35.37	24.77	37.56	27.54	40.99	19.12	27.93	20.15	29.37	23.04	32.86
STGCN	22.88	36.61	24.83	39.65	27.06	43.27	17.68	26.89	19.46	29.82	21.92	34.19
DCRNN	19.75	30.20	21.50	32.88	23.74	35.96	15.92	24.95	17.61	27.65	20.05	31.10
GWN	20.32	31.24	22.06	33.47	24.35	36.90	16.48	25.74	18.26	28.46	20.66	32.05
ASTGCN	21.61	35.02	24.16	38.98	30.19	48.21	16.48	26.08	18.43	29.34	22.26	34.78
AGCRN	20.84	32.90	23.42	36.77	29.03	46.02	16.72	26.45	18.83	29.23	22.64	33.97
MTGNN	20.54	31.72	22.78	34.48	28.36	44.79	16.58	25.82	18.46	28.44	21.62	32.28
DGCRN	19.62	30.58	21.24	32.60	25.94	38.96	15.90	25.09	17.72	27.55	20.10	30.91
GATGPT	22.52	34.10	25.86	38.47	33.82	48.05	18.39	28.55	20.84	32.58	26.08	39.51
UrbanGPT	19.32	30.16	21.02	31.79	23.18	36.10	16.24	25.52	17.93	28.26	20.68	33.84
ST-LLM+	22.35	33.88	25.96	38.57	34.13	48.48	18.47	28.66	20.95	32.71	26.25	39.69
STELA	18.69	28.53	20.08	30.65	22.56	34.54	15.64	24.61	17.28	27.32	19.46	30.93

**Figure 2: Ablation results on four datasets.**

structural guidance; (3) **w/o Fusion**: replacing the gated fusion mechanism with direct feature summation; (4) **w/o LLM**: removing the large language model adapter and predicting directly from the fused representation; (5) **Frozen LLM**: keeping the LLM fully frozen, disabling entropy-guided adaptation.

As shown in Figure 2, removing any component leads to a notable degradation across all datasets. On *NYC_subway*, excluding the transformer or graph encoder increases MAE by 6.5%–15.0%, highlighting the necessity of both temporal and structural modeling. The gated fusion also proves crucial: replacing it with simple summation consistently worsens results, confirming that adaptive weighting is essential for balancing local and global features. Finally, removing or freezing the LLM causes clear performance drops (e.g., +13.5% MAE on *NYC_subway*), demonstrating that entropy-guided selective adaptation effectively enhances contextual reasoning while maintaining stability. These results collectively verify that each module in STELA contributes complementary capabilities to the overall framework.

5.6 Model Analysis

Spatial-aware Transformer Analysis. We study the impact of stacking different numbers of layers in the spatial-aware transformer encoder. As shown in Figure 3, increasing depth from one to two layers improves performance, but deeper stacks yield limited or negative returns, likely due to redundancy with the downstream LLM encoder. This validates our choice of a lightweight two-layer design for efficient temporal abstraction.

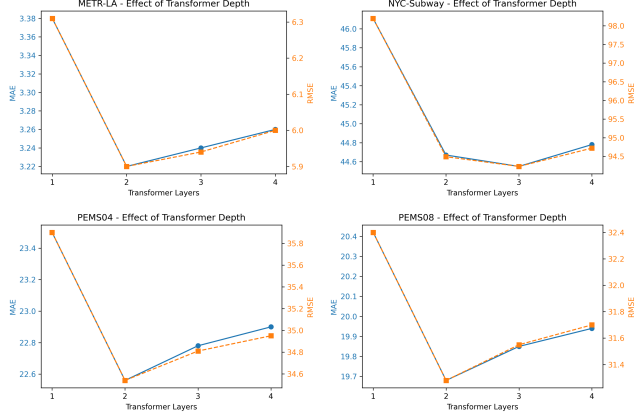


Figure 3: Effect of Transformer depth on forecasting performance.

Table 2: Graph pathway comparison (MAE / RMSE).

Dataset	Static only	Dynamic only	Fusion (ours)
METR-LA	3.51 / 6.39	3.59 / 6.58	3.22 / 5.90
NYC-Subway	49.76 / 103.2	50.11 / 104.5	44.67 / 94.69
PEMS04	23.83 / 36.2	24.14 / 36.90	22.56 / 34.54
PEMS08	20.35 / 30.49	20.64 / 31.1	19.46 / 30.93

Graph Pathway Analysis. We compare three variants of the ASTG encoder: (1) **Static only**, which uses the prior structural graph $A^{(s)}$; (2) **Dynamic only**, which uses only the learned residual graph $A^{(d)}$; and (3) **Fusion (ours)**, which combines both via a gated mixer (see Section 4.3). As shown in Table 2, the static-only variant provides strong prior but cannot adapt to transient changes, while the dynamic-only variant captures evolving dependencies but tends to overfit noisy edges. Our gated fusion consistently outperforms both, demonstrating that structural priors and adaptive residuals provide complementary benefits, improving both stability and generalization.

LLM Adaptation Strategy Analysis. We further analyze the entropy-guided fine-tuning strategy that selectively unfreezes the most task-relevant layers. Figure 4 compares four approaches under equal compute: Top-4, Bottom-4, Random-4, and our Entropy-Guided selection. Our method consistently achieves the lowest MAE and RMSE across datasets, indicating that attention entropy effectively identifies layers that contribute most to forecasting accuracy. To verify this intuition, Figure 5 visualizes layer-wise attention entropy after warm-up. Entropy varies significantly across layers (e.g., lowest at layers 10–13 for METR-LA), confirming heterogeneous attention behaviors. Low-entropy layers exhibit sharper focus patterns and encode more localized, event-driven signals, validating that selective adaptation reduces redundancy while preserving high-level reasoning.

Overall, these analyses confirm that each component in STELA is not only individually effective but also complementary, jointly

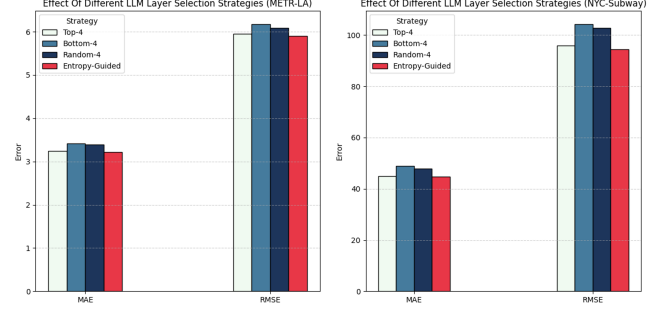


Figure 4: Comparison of four LLM fine-tuning strategies on METR-LA and NYC-Subway. Our entropy-guided strategy yields the lowest MAE and RMSE.

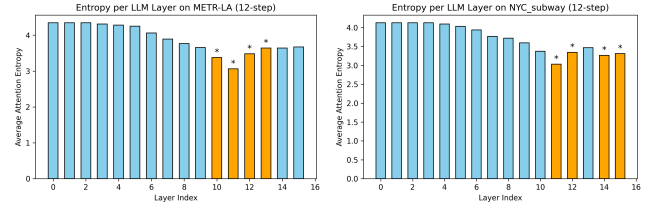


Figure 5: Layer-wise attention entropy under 12-step forecasting.

enabling scalable and interpretable forecasting across diverse spatiotemporal systems. Additional analyses are deferred to Appendix C.

6 Conclusion

We present **STELA**, a unified framework that integrates adaptive graph encoding, temporal abstraction, and semantic reasoning for long-range spatiotemporal forecasting. By combining spatial-aware Transformers with entropy-guided LLM adaptation, STELA effectively captures evolving dynamics across diverse real-world systems such as traffic, mobility, and infrastructure networks. Extensive experiments on four public benchmarks demonstrate that STELA consistently outperforms both traditional graph-based and recent LLM-enhanced forecasting methods in terms of accuracy, robustness, and scalability. Further analyses reveal that its selective fine-tuning mechanism and modular design not only improve computational efficiency but also enhance interpretability, underscoring the potential of LLM-driven reasoning for structured prediction and real-world decision-making applications.

7 Acknowledgments

This research is supported by the National Natural Science Foundation of China (Grant no. U24A20240, no. 62441226) and the ‘111 Center’ (B16037). Ziqi Xu is supported by the research support package from the School of Computing Technologies at RMIT University.

References

- [1] Lei Bai, Lina Yao, Can Li, Xianzhi Wang, and Can Wang. 2020. Adaptive graph convolutional recurrent network for traffic forecasting. *Advances in neural information processing systems* 33 (2020), 17804–17815.
- [2] Defu Cao, Furong Jia, Sercan O Arik, Tomas Pfister, Yixiang Zheng, Wen Ye, and Yan Liu. 2023. Tempo: Prompt-based generative pre-trained transformer for time series forecasting. *arXiv preprint arXiv:2310.04948* (2023).
- [3] Jianguo Chen, Kenli Li, Keqin Li, Philip S Yu, and Zeng Zeng. 2021. Dynamic planning of bicycle stations in dockless public bicycle-sharing system using gated graph neural network. *ACM Transactions on Intelligent Systems and Technology (TIST)* 12, 2 (2021), 1–22.
- [4] Yakun Chen, Xianzhi Wang, and Guandong Xu. 2023. Gatgpt: A pre-trained large language model with graph attention network for spatiotemporal imputation. *arXiv preprint arXiv:2311.14332* (2023).
- [5] Souvik Das, Lifeng Jin, Linfeng Song, Haitao Mi, Baolin Peng, and Dong Yu. 2024. Entropy guided extrapolative decoding to improve factuality in large language models. *arXiv preprint arXiv:2404.09338* (2024).
- [6] Shengnan Guo, Yilun Lin, Nan Feng, Chao Song, and Huaiyu Wan. 2019. Attention-based Spatial-Temporal Graph Convolutional Networks for Traffic Flow Forecasting. *AAAI Conference on Artificial Intelligence* 33 (2019), 922–929.
- [7] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. 2022. Lora: Low-rank adaptation of large language models. *ICLR* 1, 2 (2022), 3.
- [8] Ming Jin, Shiyu Wang, Lintao Ma, Zhixuan Chu, James Y. Zhang, Xiaoming Shi, Pin-Yu Chen, Yuxuan Liang, Yuan-Fang Li, Shirui Pan, and Qingsong Wen. 2024. Time-LLM: Time Series Forecasting by Reprogramming Large Language Models. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*.
- [9] Shiyong Lan, Yitong Ma, Weikang Huang, Wenwu Wang, Hongyu Yang, and Pyang Li. 2022. Dstagnn: Dynamic spatial-temporal aware graph neural network for traffic flow forecasting. In *International conference on machine learning*. PMLR, 11906–11917.
- [10] Fuxian Li, Jie Feng, Huan Yan, Guangyin Jin, Fan Yang, Funing Sun, Depeng Jin, and Yong Li. 2023. Dynamic graph convolutional recurrent network for traffic prediction: Benchmark and solution. *ACM Transactions on Knowledge Discovery from Data* 17, 1 (2023), 1–21.
- [11] Yue Li, Shuai Guo, Qipeng Song, Yao Wang, Xiaomin Wei, and Jianfeng Ma. 2023. Mapping prediction with recurrent neural networks for future lisp enabled networks. *Journal of Information and Intelligence* 1, 2 (2023), 134–147.
- [12] Yaguang Li, Rose Yu, Cyrus Shahabi, and Yan Liu. 2018. Diffusion Convolutional Recurrent Neural Network: Data-Driven Traffic Forecasting. In *International Conference on Learning Representations (ICLR)*.
- [13] Zhonghang Li, Lianghao Xia, Jiabin Tang, Yong Xu, Lei Shi, Long Xia, Dawei Yin, and Chao Huang. 2024. UrbanGPT: Spatio-Temporal Large Language Models. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '24)*. ACM.
- [14] Sihao Lin, Pumeng Lyu, Dongrui Liu, Tao Tang, Xiaodan Liang, Andy Song, and Xiaojun Chang. 2024. Mlp can be a good transformer learner. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 19489–19498.
- [15] Chenxi Liu, Kethmi Hirushini Hettige, Qianxiang Xu, Cheng Long, Shili Xiang, Gao Cong, Ziyue Li, and Rui Zhao. 2025. ST-LLM+: Graph Enhanced Spatio-Temporal Large Language Models for Traffic Prediction. *IEEE Transactions on Knowledge and Data Engineering* (2025).
- [16] Kevin Lu, Aditya Grover, Pieter Abbeel, and Igor Mordatch. 2022. Frozen pre-trained transformers as universal computation engines. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 36. 7628–7636.
- [17] Sathika Malladi, Tianyu Gao, Eshaan Nichani, Alex Damian, Jason D Lee, Danqi Chen, and Sanjeev Arora. 2023. Fine-tuning language models with just forward passes. *Advances in Neural Information Processing Systems* 36 (2023), 53038–53075.
- [18] Julien Monteil, Anton Dekusar, Claudio Gambella, Yassine Lassoued, and Martin Mevisen. 2021. On model selection for scalable time series forecasting in transport networks. *IEEE Transactions on Intelligent Transportation Systems* 23, 7 (2021), 6699–6708.
- [19] Arijit Nag, Bidisha Samanta, Animesh Mukherjee, Niloy Ganguly, and Soumen Chakrabarti. 2023. Entropy-guided vocabulary augmentation of multilingual language models for low-resource tasks. In *Findings of the Association for Computational Linguistics: ACL*. 8619–8629.
- [20] Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. 2023. A Time Series is Worth 64 Words: Long-term Forecasting with Transformers. In *International Conference on Learning Representations*.
- [21] Adam Paryl, Grzegorz Rypeś, Grzegorz Kurzejanski, Bartosz Zieliński, and Tomasz Trzcinski. 2023. Active Visual Exploration Based on Attention-Map Entropy. In *32nd International Joint Conference on Artificial Intelligence*. International Joint Conferences on Artificial Intelligence.
- [22] Yanjun Qin, Yuchen Fang, Haiyong Luo, Fang Zhao, and Chenxing Wang. 2021. DMGCRN: Dynamic multi-graph convolution recurrent network for traffic forecasting. *arXiv preprint arXiv:2112.02264* (2021).
- [23] Yilong Ren, Yue Chen, Shuai Liu, Boyue Wang, Haiyang Yu, and Zhiyong Cui. 2024. TPLLM: A Traffic Prediction Framework Based on Pretrained Large Language Models. *arXiv preprint arXiv:2403.02221* (2024).
- [24] Zheng Shi, Yingjun Zhang, Jingping Wang, Jiahu Qin, Xiaoqian Liu, Hui Yin, and Hua Huang. 2023. DAGCRN: Graph convolutional recurrent network for traffic forecasting with dynamic adjacency matrix. *Expert Systems with Applications* 227 (2023), 120259.
- [25] Chao Song, Youfang Lin, Shengnan Guo, and Huaiyu Wan. 2020. Spatial-temporal synchronous graph convolutional networks: A new framework for spatial-temporal network data forecasting. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 34. 914–921.
- [26] Cheng Sun, Yuxuan Li, Hengyu Li, and Seonghoon Hong. 2024. TEST: Text Prototype Aligned Embedding to Activate LLM's Ability for Time Series. In *Proceedings of the Twelfth International Conference on Learning Representations (ICLR)*.
- [27] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmin Babaei, Nikolay Bashlykov, Soumya Batra, Prajwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).
- [28] Daheng Wang, Meng Jiang, Munira Syed, Oliver Conway, Vishal Juneja, Sriram Subramanian, and Nitesh V Chawla. 2020. Calendar graph neural networks for modeling time structures in spatiotemporal user behaviors. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. 2581–2589.
- [29] Wei Wang, Junyang Chen, Yushu Zhang, Zhiguo Gong, Neeraj Kumar, and Wei Wei. 2021. A multi-graph convolutional network framework for tourist flow prediction. *ACM Transactions on Internet Technology (TOIT)* 21, 4 (2021), 1–13.
- [30] Haixu Wu, Yize Xia, Zongxin Wang, Haoyi Tan, Yuliang Lin, and Huijuan Zhao. 2021. Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. In *Advances in Neural Information Processing Systems*, Vol. 34. 22419–22430.
- [31] Zonghan Wu, Shirui Pan, Guodong Long, Jing Jiang, Chengqi Chang, and Philipp Zhang. 2019. Graph WaveNet for Deep Spatial-Temporal Graph Modeling. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI)*. International Joint Conferences on Artificial Intelligence, Macao, China, 1907–1913. doi:10.24963/ijcai.2019/264
- [32] Zonghan Wu, Shirui Pan, Guodong Long, Jing Jiang, Xiaojun Chang, and Chengqi Zhang. 2020. Connecting the Dots: Multivariate Time Series Forecasting with Graph Neural Networks. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*.
- [33] Weijian Xu, Jingjin Liu, Jingwen Yan, Juan Yang, Huifeng Liu, and Teng Zhou. 2023. Dynamic spatiotemporal graph wavelet network for traffic flow prediction. *IEEE Internet of Things Journal* 11, 5 (2023), 8019–8029.
- [34] Jiexia Ye, Juanjuan Zhao, Kejiang Ye, and Chengzhong Xu. 2020. How to build a graph-based deep learning architecture in traffic domain: A survey. *IEEE Transactions on Intelligent Transportation Systems* 23, 5 (2020), 3904–3924.
- [35] Bing Yu, Haoqiang Yin, and Zhanxing Zhu. 2018. Spatio-Temporal Graph Convolutional Networks: A Deep Learning Framework for Traffic Forecasting. In *Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence, IJCAI-18*. 3634–3640.
- [36] Haitao Yuan and Guoliang Li. 2021. A survey of traffic prediction: from spatiotemporal data to intelligent transportation. *Data Science and Engineering* 6, 1 (2021), 63–85.
- [37] Yuan Yuan, Jingtao Ding, Jie Feng, Depeng Jin, and Yong Li. 2024. Unist: A prompt-empowered universal model for urban spatio-temporal prediction. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4095–4106.
- [38] Haoran Zhang, Jinyu Chen, Wenjing Li, Xuan Song, and Ryosuke Shibasaki. 2020. Mobile phone GPS data in urban ride-sharing: An assessment method for emission reduction potential. *Applied Energy* 269 (2020), 115038.
- [39] Zijian Zhang, Yujie Sun, Zepu Wang, Yuqi Nie, Xiaobo Ma, Peng Sun, and Ruolin Li. 2024. Large language models for mobility in transportation systems: A survey on forecasting tasks. *arXiv preprint arXiv:2405.02357* (2024).
- [40] Zhisong Zhang, Yan Wang, Xinting Huang, Tianqing Fang, Hongming Zhang, Chenlong Deng, Shuaiyi Li, and Dong Yu. 2025. Attention entropy is a key factor: An analysis of parallel context encoding with full-attention-based pre-trained language models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 9840–9855.
- [41] Ling Zhao, Yujiao Song, Chao Zhang, Yu Liu, Pu Wang, Tao Lin, and Min Deng. 2019. T-GCN: A Temporal Graph Convolutional Network for Traffic Prediction. *IEEE Transactions on Intelligent Transportation Systems* 21, 9 (2019), 3848–3858.
- [42] Sichen Zhao, Wei Shao, Jeffrey Chan, Ziqi Xu, and Flora Salim. 2025. FairDRL-ST: Disentangled Representation Learning for Fair Spatio-Temporal Mobility Prediction. In *Proceedings of the 33rd ACM International Conference on Advances in Geographic Information Systems, SIGSPATIAL*. 103–106. doi:10.1145/3748636.3762713
- [43] Chuanpan Zheng, Xiaoliang Fan, Cheng Wang, and Jianzhong Qi. 2020. GMAN: A Graph Multi-Attention Network for Traffic Prediction. *AAAI Conference on*

Algorithm 1: The STELA Framework.

Input: Historical traffic data X_P over P past time steps, adjacency matrix A , pretrained LLM with L layers, and all hyperparameters.

Output: Trained STELA model.

for each epoch do

Shuffle training data;

for each batch X_P in training data do

$H^{(t)} \leftarrow$ Spatial-Aware Transformer Encoder by Eqs. (1)–(4);

$H^{(g)} \leftarrow$ Adaptive Spatiotemporal Graph Encoder by Eqs. (5)–(9);

$H^{(f)} \leftarrow$ Multi-Path Feature Fusion by Eqs. (10)–(13);

Compute aligned semantic features $H^{(\text{align})}$ by Eq. (14);

Initialize entropy-guided LLM adapter; compute H_i and select U lowest-entropy layers S ;

for $i = 1$ to L do

if $i \in S$ then

Update LLM layer i with $H^{(\text{align})}$;

else

Freeze layer i parameters;

$H^{(\text{LLM})} \leftarrow \text{LLM}_{\text{partial}}(H^{(\text{align})})$;

Predict $\hat{Y}_S$ by the linear head ;

Update all trainable parameters by minimizing MAE loss $\mathcal{L}$ via back-propagation;

Artificial Intelligence 34, 1 (2020), 1234–1241.

- [44] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. 2021. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 35. 11106–11115.

A Algorithm of STELA

Algorithm 1 summarizes the overall training and inference pipeline of STELA. The framework integrates four major modules: (1) the Spatial-Aware Transformer Encoder (STFormer) for temporal summarization with cross-node attention, (2) the Adaptive Spatiotemporal Graph Encoder (ASTG) for dynamic relation modeling, (3) the Multi-Path Feature Fusion module for gated integration of multi-scale features, and (4) the Entropy-Guided LLM Adapter for semantic reasoning and context-aware refinement.

B Additional Experimental Details

B.1 Implementation Details

All experiments are implemented in PyTorch 2.2.2 with CUDA 12.2. The LLM backbone is LLaMA-7B, integrated through the HuggingFace Transformers(v4.31.0) library in a Python 3.11 environment. For all models, mixed-precision (BF16) training is enabled to improve efficiency. We adopt the Adam optimizer with an initial learning rate of $1e-4$, decayed by a factor of 0.5 every 10 epochs if validation loss does not improve. The model is optimized with mean absolute error (MAE) loss, and we report results using Mean

Table 3: Detailed statistics of the four datasets used in our experiments.

Dataset	#Nodes	#Timestamps	Time Granularity
METR-LA	207	34,272	5 minutes
PEMS04	307	16,992	5 minutes
PEMS08	170	17,856	5 minutes
NYC-Subway	167	4,464	1 hour

Absolute Error and Root Mean Squared Error (RMSE), defined as:

$$\text{MAE} = \frac{1}{NT} \sum_{i=1}^N \sum_{t=1}^T |\hat{y}_{i,t} - y_{i,t}|, \quad (18)$$

$$\text{RMSE} = \sqrt{\frac{1}{NT} \sum_{i=1}^N \sum_{t=1}^T (\hat{y}_{i,t} - y_{i,t})^2}, \quad (19)$$

where N and T denote the number of nodes and forecasting steps, respectively, and $\hat{y}_{i,t}$ and $y_{i,t}$ represent predicted and ground-truth values.

Each model is trained for 100 epochs with a batch size of 4 and early stopping patience of 10. Dropout is set to 0.1 in all layers. All experiments are conducted on a server equipped with NVIDIA A800 GPUs (80 GB memory). Training one full model on METR-LA takes approximately 4.5 hours. We fix random seeds across NumPy and PyTorch to ensure reproducibility. Model weights are initialized with Xavier uniform initialization. The implementation of STELA will be released publicly after acceptance.

B.2 Dataset Details

We use four benchmark spatiotemporal datasets: **METR-LA**, **PEMS04**, **PEMS08**, and **NYC-Subway**. Their basic statistics are summarized in Table 3, and detailed descriptions are provided below.

- **METR-LA** Collected by the Caltrans Performance Measurement System (PeMS), METR-LA records average traffic speed from 207 loop detectors on Los Angeles County highways. Data were collected every 5 minutes from March to June 2012.
- **PEMS04 and PEMS08**. PEMS04 is another PeMS dataset, consisting of traffic flow readings from 307 sensors in District 4 (Bay Area). It covers January to February 2018 with 5-minute intervals. PEMS08 contains traffic flow data from 170 detectors in District 8 (San Bernardino area) during July–August 2016.
- **NYC-Subway**. NYC-Subway is built from the Metropolitan Transportation Authority (MTA) open data portal,¹ which records hourly turnstile entries at subway stations across New York City. We filtered and reformatted the raw data to retain 167 stations in Queens and Brooklyn from July to December 2024, ensuring consistent temporal alignment and removing incomplete records. Edges in the graph are defined by subway line connectivity, resulting in a sparse but physically interpretable topology. We apply log-scaling to ridership counts and linearly interpolate short missing intervals.

¹https://data.ny.gov/Transportation/MTA-Subway-Hourly-Ridership-2020-2024/wujg-7c2s/about_data

Preprocessing Summary. All datasets are framed as sequence-to-sequence prediction tasks with 12 input and 12 output steps. Each sequence is normalized using statistics computed from the training set only to avoid data leakage. Missing values are linearly interpolated when possible and otherwise masked during training. We adopt a chronological split with 70%, 15%, and 15% of samples used for training, validation, and testing, respectively. This ratio ensures sufficient temporal coverage for model learning while maintaining reliable evaluation of generalization performance across different time periods.

B.3 Baseline Details

We compare STELA with a wide range of baselines, including statistical models, spatiotemporal graph networks, and recent large-model-based approaches.

- **HA:** Historical average method that predicts by averaging past observations at the same time of day.
- **ARIMA:** A classical autoregressive model capturing linear temporal dependencies.
- **TGCN** [41]: Combines graph convolution with GRU to jointly model spatial and temporal dependencies.
- **STGCN** [35]: Employs Chebyshev graph convolution and gated 1D convolution for efficient spatiotemporal learning.
- **DCRNN** [12]: Models directed spatial diffusion using random walks integrated with GRU cells.
- **Graph WaveNet** [31]: Learns adaptive adjacency and applies dilated convolutions for long-range temporal modeling.
- **ASTGCN** [6]: Introduces spatial, temporal, and semantic attention for interpretable spatiotemporal forecasting.
- **AGCRN** [1]: Learns node-specific embeddings to construct data-driven adaptive graphs.
- **MTGNN** [32]: Integrates graph learning with temporal convolution to capture dynamic dependencies.
- **DGCRN** [10]: Models static and dynamic spatial relations through dual diffusion pathways.
- **GATGPT** [4]: Combines graph attention networks with GPT for semantic temporal modeling.
- **UrbanGPT** [13]: A spatio-temporal large language model that integrates a temporal dependency encoder with instruction-tuned LLMs, enabling cross-city generalization and zero-shot forecasting under data scarcity.
- **ST-LLM+** [15]: A graph-enhanced large language model that integrates a partially frozen graph attention module and LoRA fine-tuning to efficiently adapt pretrained LLMs for traffic prediction.

C Extended Experimental Analysis

C.1 Training Efficiency Analysis

Before conducting efficiency analysis, we note that the LLaMA-7B backbone originally consists of 32 Transformer layers. For practical efficiency, we adopt the first 16 layers as the usable backbone in all our experiments. This allows us to explore fine-tuning configurations under moderate computational budgets while preserving sufficient model capacity.

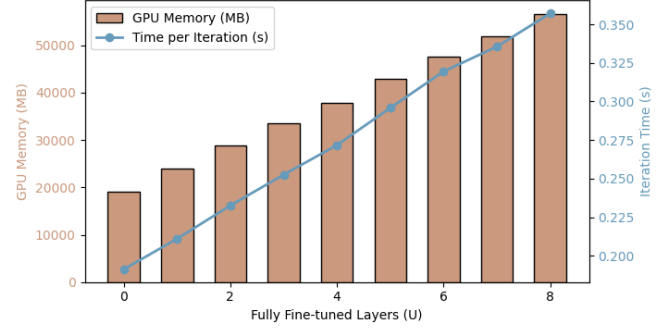


Figure 6: GPU memory usage and iteration time under different numbers of fine-tuned LLM layers (U), with METR-LA. Both grow nearly linearly, indicating efficient scalability of the proposed partial fine-tuning strategy.

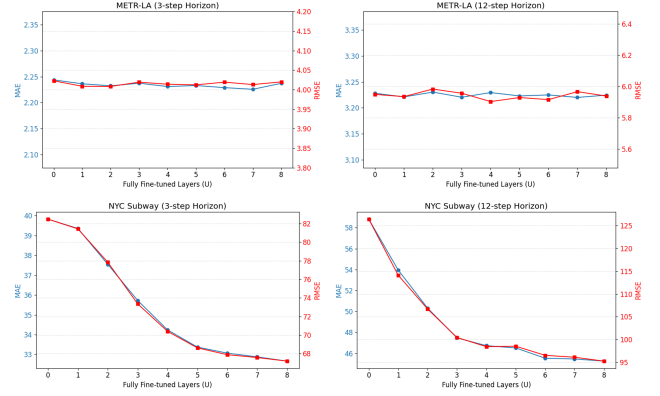


Figure 7: Forecasting accuracy (MAE and RMSE) under different numbers of fine-tuned LLM layers (U). Accuracy improves rapidly in the first few layers and plateaus beyond $U = 4-6$, suggesting that moderate adaptation depth offers the best balance between accuracy and efficiency.

To further examine the computational efficiency of our selective fine-tuning strategy, we measure GPU memory usage and per-iteration time when varying the number of fine-tuned LLM layers (U), using METR-LA, the number of nodes is 207. As shown in Figure 6, both metrics grow almost linearly with U , demonstrating that the proposed entropy-guided partial fine-tuning achieves favorable scalability under limited GPU budgets. Specifically, GPU memory consumption increases from 19.1 GB to 56.6 GB as U grows from 0 to 8, while iteration time rises moderately from 0.19 s to 0.36 s. This confirms that STELA maintains high training efficiency without sacrificing flexibility in model adaptation.

C.2 Forecasting Accuracy vs. Fine-tuning Depth

To further analyze the impact of fine-tuning depth on predictive performance, we vary the number of unfrozen LLM layers (U) from 0 to 8 while keeping the total backbone depth fixed at 16. As

illustrated in Figure 7, the forecasting accuracy improves steadily as U increases in the early stage, but the gains diminish beyond moderate adaptation levels.

For short-term forecasting (e.g., $T = 3$), the error reduction is most pronounced when unfreezing the first few layers, after which performance stabilizes and even exhibits slight oscillations. This suggests that shallow adaptation already captures the most relevant temporal and spatial semantics for near-future prediction, while deeper tuning may introduce redundancy or overfitting. For longer forecasting horizons (e.g., $T = 12$), the overall improvement trend remains similar—accuracy increases consistently as U grows, but the marginal gain beyond the mid-depth range becomes limited.

Overall, these results indicate that moderate adaptation depth provides the best trade-off between model expressiveness and computational cost. Empirically, we find that unfreezing $U = 4$ layers yields a near-optimal balance, achieving stable accuracy improvements while maintaining training efficiency.

C.3 Station-level Case Study

To further assess STELA’s capability in capturing heterogeneous mobility dynamics, we visualize forecasting results from three representative NYC-Subway stations: **57 St–7 Av (Station 9)**, **181 St (Station 146)**, and **34 St–Penn Station (Station 164)**. These locations reflect distinct spatial functions—a commercial hub, a residential periphery, and a major interchange—covering a broad spectrum of flow intensity and temporal regularity.

As shown in Figure 8, STELA effectively adapts its forecasting behavior to the unique temporal and structural patterns of each region. At 57 St–7 Av, where ridership follows a clear workday rhythm, the model aligns tightly with observed commuter peaks, reflecting the strength of the spatial-aware Transformer encoder in maintaining global temporal consistency while selectively attending to correlated stations. At 181 St, located in a quieter residential area with less regular patterns, the adaptive spatiotemporal graph encoder stabilizes predictions by dynamically reweighting nearby nodes, preventing overfitting to short-term fluctuations. Meanwhile, at 34 St–Penn Station—a high-volume interchange with rapid traffic transitions—the entropy-guided LLM adapter introduces contextual awareness, allowing the model to anticipate demand surges around events or transfer periods without explicit event labels.

Overall, these qualitative results demonstrate that STELA’s design effectively integrates temporal, structural, and semantic cues into a unified representation. The combination of adaptive topology and selective semantic adaptation enables robust forecasting under both periodic and context-driven variations, confirming that the proposed architecture not only learns correlations within time and space, but also interprets broader behavioral semantics across the urban network.

D Limitations and Future Work

Although STELA achieves consistent improvements across multiple datasets and exhibits strong semantic adaptability, several limitations remain to be addressed in future research. First, the current entropy-guided fine-tuning mechanism selects layers based solely on attention entropy measured during a warm-up stage. While

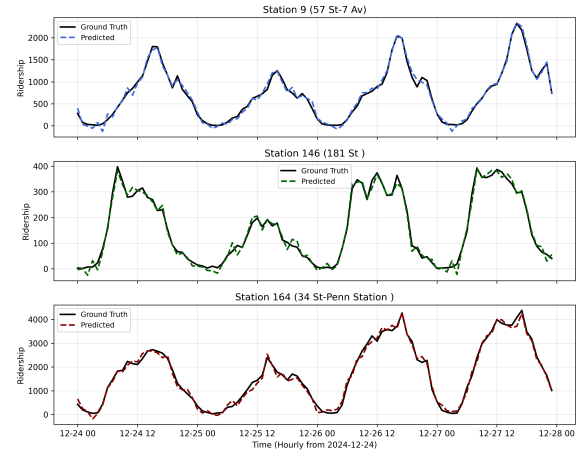


Figure 8: Station-level forecasting trends on NYC-Subway data. The figure illustrates three representative stations—57 St–7 Av (downtown hub), 181 St (residential area), and 34 St–Penn Station (major interchange)—where STELA’s predictions (dashed) align closely with real ridership trends (solid) across heterogeneous spatial contexts.

effective, this heuristic may not fully capture the dynamic relevance of different layers throughout training, and more adaptive or learnable selection strategies could further enhance efficiency and robustness. Second, STELA primarily relies on structured numerical inputs and pre-trained LLM semantics. Incorporating richer multimodal signals—such as text-based event reports, traffic camera feeds, or social media activity—could provide more fine-grained context for event-driven forecasting. Third, the framework’s reliance on large-scale language models introduces a nontrivial computational cost during training, which may limit real-time deployment in resource-constrained environments. Exploring lightweight adapter architectures or quantization-aware fine-tuning could mitigate this issue.

In future work, we plan to further optimize the efficiency of STELA’s semantic adaptation process. Although the entropy-guided fine-tuning mechanism already minimizes redundant updates, the integration of large-scale LLMs still incurs nontrivial memory and computational overhead. To alleviate this, we will explore parameter-efficient fine-tuning strategies such as Low-Rank Adaptation (LoRA), lightweight adapter modules, and mixed-precision quantization, which can significantly reduce the number of trainable parameters while maintaining comparable accuracy. We also intend to investigate hybrid optimization schemes that dynamically balance frozen and trainable layers during training, enabling adaptive resource allocation without sacrificing forecasting precision. These efforts aim to make STELA more scalable and deployable in real-time or resource-limited environments.